

1. Performance Testing Lifecycle & SOP (Standard Operating Procedure)

This becomes your go-to guide for how performance testing is done at Ascensus — clear steps, roles, tools, best practices, and success criteria.

1. Performance Testing Lifecycle Overview

What is Performance Testing?

Performance Testing ensures applications can scale and perform under expected (and unexpected) load with optimal responsiveness, throughput, and resource usage.

Lifecycle Phases

Phase	Description	Output
1. Request Initiation	Business/QA initiates a performance test request.	Email or Jira ticket
2. Requirements Gathering	We collect test goals, expected load, data size, env info.	Req Doc
3. Planning & Scripting	Build test scripts (JMeter, YAML), define workload models.	JMX/YAML
4. Baseline Run	Initial test to assess readiness and stability.	Baseline Report
5. Full Load Test	Execute under desired user loads.	Full Report
6. Analysis & Debug	Identify bottlenecks, logs, infra stats, metrics.	Root Cause Summary
7. Reporting & Sign-off	Share metrics with business/devs, document outcomes.	Summary Report
8. Retest (if needed)	Post-fix validation or regression test.	Final report

2. Stakeholder Responsibilities

- Write and maintain test scripts (JMeter/YAML)
- Configure environment access
- Execute and analyze results
- Define SLAs & expectations
- Approve report sign-offs
- Provide endpoint details, architecture flow
- Investigate and fix perf issues
- Support Rancher, Kube, New Relic, DB logs
- Enable access to relevant clusters and metrics

3. Tools & Technologies

Tool	Purpose
JMeter	Load generation
Taurus (bzt)	YAML-based test orchestration
BlazeMeter	Cloud execution and dashboards
Jenkins	CI/CD integration
New Relic	APM, server metrics
Kubernetes	Container orchestration
Splunk / Oracle Logs	Error & infra diagnostics

4. Entry & Exit Criteria

- Request submitted and approved
- Test environment confirmed ready
- Required tools installed and configured
- Access granted (Rancher, DB, APM)

- Performance test completed without critical errors
- Report reviewed by QA + Dev + Product
- Action items logged for bottlenecks

5. Required Inputs Before Starting

Category	Example
Env Details	Staging URL, cluster, DB
Load Profile	200 users, 60 min, 2 TPS
Use Cases	Login, Dashboard, Report APIs
Data Volume	100K records in test set
SLA Targets	Avg 3s, 90% 5s
Tools To Use	JMeter + Blazemeter
Contacts	Dev: Raj, QA: Shravan

6. Test Execution Standards

Type	When to Use	Duration	Notes
Baseline	Early-stage, sanity	~10 mins	Low load
Load	Main test	30–60 mins	Matches prod
Soak	Stability	4+ hrs	Memory leaks, infra
Spike	Burst check	15–20 mins	Sudden load
Stress	Capacity	Until fail	Upper threshold

7. Reporting & Communication

- Avg / 90th / 95th Response Times
- TPS (Transactions Per Second)
- Error %
- Server Metrics: CPU, memory
- JMeter/Blazemeter Graphs
- Root cause (if failure)

- Upload to Confluence + Attach PDF
- Link in Jira Story or Slack to stakeholders
- Archive in `/performance/reports/YYYY/QX`

8. Troubleshooting & FAQs

Check if `execution` block in YAML is valid; validate JMeter plugin paths.
Check DB access and connection pool limits. Use Splunk or New Relic for detailed error logs.
Export CSV and use JMeter HTML report or Taurus viewer instead.

9. Final Notes & Versioning

- Always include test version/tag in your test summary.
- Link test plan, scripts, and report in one location.
- Update this page if new tools/processes are added.

